Top 50 Coding Questions for Product based companies(including zoho)

1. Given an array of size **N-1** such that it only contains distinct integers in the range of **1 to N**. Find the missing element.

Example 1:

Input:
N = 5
A[] = {1,2,3,5}
Output: 4

Example 2:

Input:
N = 10
A[] = {6,1,2,8,3,4,7,10,5}
Output: 9

Expected Time Complexity: O(N)
Expected Auxiliary Space: O(1)

Constraints:
$1 \le N \le 10^6$
$1 \le A[i] \le 10^6$

2. Given an integer n, write a function that returns count of trailing zeroes in n!.

Examples :

Input: *n = 5*
Output: *1*
Explanation: *Factorial of 5 is 120 which has one trailing 0.*
Input: *n = 20*
Output: *4*
Explanation: *Factorial of 20 is 2432902008176640000 which has 4 trailing zeroes.*
Input: *n = 100*
Output: *24*

2. Given two integers representing the Numerator and Denominator of a fraction, return the fraction in string format. If the fractional part is repeating, enclose the repeating part in parentheses.

Examples:
Input: Numerator = 1, Denominator = 2
Output: "0.5"
1/2 = 0.5 with no repeating part.

Input: Numerator = 50, Denominator = 22
Output: "2.(27)"
50/22 = 2.27272727... Since fractional part (27)
is repeating, it is enclosed in parentheses.

3. Given a positive integer **N**, the task is to find the **N**th natural number after removing all the natural numbers containing digit **9**.

Examples:
Input: *N = 8*
Output: *8*
Explanation:
Since 9 is the first natural number that contains the digit 9 and is the 9th natural number, therefore, no removal required to find the 8th natural number, which is 8.
Input: *N = 9*
Output: *10*
Explanation:
Removing number 9, the first 9 natural numbers are {1, 2, 3, 4, 5, 6, 7, 8, 10}. Therefore, the 9th natural number is 10.

4. Given an array of positive numbers, find the smallest positive integer value that cannot be represented as the sum of elements of any subset of a given set.
   The expected time complexity is O(nlogn).

Examples:
Input: arr[] = {1, 10, 3, 11, 6, 15};
Output: 2
Input: arr[] = {1, 1, 1, 1};
Output: 5
Input: arr[] = {1, 1, 3, 4};
Output: 10
Input: arr[] = {1, 2, 5, 10, 20, 40};
Output: 4
Input: arr[] = {1, 2, 3, 4, 5, 6};
Output: 22

5. Given an array of integers **arr[]** of size **N** and an integer, the task is to rotate the array elements to the **left** by **d** positions.

Examples:
Input:
arr[] = {1, 2, 3, 4, 5, 6, 7}, d = 2
Output: *3 4 5 6 7 1 2*
Input: *arr[] = {3, 4, 5, 6, 7, 1, 2}, d=2*
Output: *5 6 7 1 2 3 4*

6. Find the majority element in the array. A **majority element** in an array A[] of size n is an element that appears more than n/2 times (and hence there is at most one such element).

Examples :
Input : *A[]={3, 3, 4, 2, 4, 4, 2, 4, 4}*
Output : *4*
Explanation: *The frequency of 4 is 5 which is greater than the half of the size of the array size.*
Input : *A[] = {3, 3, 4, 2, 4, 4, 2, 4}*
Output : *No Majority Element*

Explanation: *There is no element whose frequency is greater than the half of the size of the array size.*

7. Given a non-negative number represented as an array of digits, add 1 to the number (increment the number represented by the digits). The digits are stored such that the most significant digit is the first element of the array.

Examples :
Input : [1, 2, 4]

Output : [1, 2, 5]

Input : [9, 9, 9]

Output : [1, 0, 0, 0]

8. Given an array of positive and negative numbers, arrange them in an alternate fashion such that every positive number is followed by negative and vice-versa. Order of elements in output doesn't matter. Extra positive or negative elements should be moved to end.

Examples:
Input :
arr[] = {-2, 3, 4, -1}
Output :
arr[] = {-2, 3, -1, 4} OR {-1, 3, -2, 4} OR ..

Input :
arr[] = {-2, 3, 1}
Output :
arr[] = {-2, 3, 1} OR {-2, 1, 3}

Input :
arr[] = {-5, 3, 4, 5, -6, -2, 8, 9, -1, -4}
Output :
arr[] = {-5, 3, -2, 5, -6, 4, -4, 9, -1, 8}

9. Find the factorial of a large number.
   Input: *100*
   Output: *93326215443944152681699238856266700490715968264381621468592963895217599993229915608941463976156518286253697920827223758251185210916864000000000000000000000000*

10. Given two sorted arrays of size m and n respectively, you are tasked with finding the element that would be at the k'th position of the final sorted array.

Examples:
Input : Array 1 - 2 3 6 7 9
Array 2 - 1 4 8 10
k = 5
Output : 6
Explanation: The final sorted array would be -
1, 2, 3, 4, 6, 7, 8, 9, 10
The 5th element of this array is 6.

Input : Array 1 - 100 112 256 349 770
Array 2 - 72 86 113 119 265 445 892
k = 7
Output : 256
Explanation: Final sorted array is -
72, 86, 100, 112, 113, 119, 256, 265, 349, 445, 770, 892
7th element of this array is 256.

11. Given a string, the task is to reverse the order of the words in the given string.
Examples:
Input: *s = "geeks quiz practice code"*
Output: *s = "code practice quiz geeks"*
Input: *s = "i love programming very much"*
Output: *s = "much very programming love i"*

12. Given a Roman numeral, the task is to find its corresponding decimal value.

Example :
Input: *IX*
Output: *9*
Explanation: *IX is a Roman symbol which represents 9*
Input: *XL*
Output: *40*
Explanation: *XL is a Roman symbol which represents 40*

13. Write a program to Validate an IPv4 Address.

IPv4 Address *are canonically represented in dot-decimal notation, which consists of four decimal numbers, each ranging from 0 to 255, separated by dots, e.g., 172.16.254.1 .*
A **valid IPv4 Address** is of the form **x1.x2.x3.x4** where **0 <= (x1, x2, x3, x4) <= 255**.
Thus, we can write the generalized form of an IPv4 address as (0-255).(0-255).(0-255).(0-255).

Example 1:

Input:
IPv4 address = 222.111.111.111
Output: 1
Explanation: Here, the IPv4 address is as per the criteria mentioned and also all four decimal numbers lies in the mentioned range.

Example 2:

Input:
IPv4 address = 5555..555
Output: 0

Explanation: 5555..555 is not a valid IPv4 address, as the middle two portions are missing.

14. Given an array A[] of n numbers and another number x, the task is to check whether or not there exist two elements in A[] whose sum is exactly x.

Examples:
Input: arr[] = {0, -1, 2, -3, 1}, x= -2
Output: Yes
Explanation: If we calculate the sum of the output,1 + (-3) = -2
Input: arr[] = {1, -2, 1, 0, 5}, x = 0
Output: No

15. Given two unsorted arrays that represent two sets (elements in every array are distinct), find the union and intersection of two arrays.

Example:
arr1[] = {7, 1, 5, 2, 3, 6}

arr2[] = {3, 8, 6, 20, 7}

Then your program should print Union as {1, 2, 3, 5, 6, 7, 8, 20} and Intersection as {3, 6, 7}. Note that the elements of union and intersection can be printed in any order.

16. Given a string **str**, find the length of the longest substring without repeating characters.

Example:
Example 1:
Input: *"ABCDEFGABEF"*
Output: *7*
Explanation: *The longest substring without repeating characters are "ABCDEFG", "BCDEFGA", and "CDEFGAB" with lengths of 7*
Example 2:
Input: *"GEEKSFORGEEKS"*
Output: *7*

Explanation: *The longest substrings without repeating characters are "EKSFORG" and "KSFORGE", with lengths of 7*

17. Given a pointer to the head node of a linked list, the task is to reverse the linked list. We need to reverse the list by changing the links between nodes.

Examples:
Input: *Head of following linked list*
1->2->3->4->NULL
Output: *Linked list should be changed to,*
4->3->2->1->NULL
Input: *Head of following linked list*
1->2->3->4->5->NULL
Output: *Linked list should be changed to,*
5->4->3->2->1->NULL

18. Given Pointer/Reference to the head of the linked list, the task is to **Sort the given linked list using Merge Sort**.
Note: If the length of linked list is odd, then the extra node should go in the first list while splitting.

Example 1:

Input:
N = 5
value[] = {3,5,2,4,1}
Output: 1 2 3 4 5
Explanation: After sorting the given linked list, the resultant matrix will be 1->2->3->4->5.

19. Given an infix expression in the form of string **str**. Convert this infix expression to postfix expression.

- **Infix expression:** The expression of the form a **op** b. When an operator is in-between every pair of operands.

- **Postfix expression:** The expression of the form a b **op**. When an operator is followed for every pair of operands.
  Note: The order of precedence is: ^ **greater than** * **equals to** / **greater than** + **equals to** -.

Example 1:

Input: str = "a+b*(c^d-e)^(f+g*h)-i"
Output: abcd^e-fgh*+^*+i-
Explanation:
After converting the infix expression
into postfix expression, the resultant
expression will be abcd^e-fgh*+^*+i-

20. Given a string **s** and an integer **k**, the task is to reduce the string by applying the following operation:
Choose a group of **k** consecutive identical characters and remove them.

The operation can be performed any number of times until it is no longer possible.

Example 1:

Input:
k = 2
s = "geeksforgeeks"
Output:
gksforgks
Explanation:
Modified String after each step:
"g**ee**ksforg**ee**ks" -> "gksforgks"

21. Suppose there is a circle. There are **N** petrol pumps on that circle. You will be given two sets of data.
1. The amount of petrol that every petrol pump has.
2. Distance from that petrol pump to the next petrol pump.
Find a starting point where the truck can start to get through the complete circle without exhausting its petrol in between.
Note: Assume for 1 litre petrol, the truck can go 1 unit of distance.

Example 1:

Input:
N = 4
Petrol = 4 6 7 4
Distance = 6 5 3 5
Output: 1
Explanation: There are 4 petrol pumps with amount of petrol and distance to next petrol pump value pairs as {4, 6}, {6, 5}, {7, 3} and {4, 5}. The first point from where truck can make a circular tour is 2nd petrol pump. Output in this case is 1 (index of 2nd petrol pump).

Your Task:
Your task is to complete the function **tour**() which takes the required data as inputs and returns an integer denoting a point from where a truck will be able to complete the circle (The truck will stop at each petrol pump and it has infinite capacity). If there exists multiple such starting points, then the function must return the first one out of those. (return -1 otherwise)

Expected Time Complexity: O(N)
Expected Auxiliary Space : O(1)

Constraints:

$2 \leq N \leq 10000$

$1 \leq$ petrol, distance ≤ 1000

22. A celebrity is a person who is known to all but does not know anyone at a party. If you go to a party of N people, find if there is a celebrity in the party or not.
A square NxN matrix M[][] is used to represent people at the party such that if an element of row i and column j is set to 1 it means ith person knows jth person. Here M[i][i] will always be 0.
Note: Follow 0 based indexing.
Follow Up: Can you optimize it to O(N)

Example 1:

Input:
N = 3
M[][] = {{0 1 0},
{0 0 0},
{0 1 0}}
Output: 1
Explanation: 0th and 2nd person both know 1. Therefore, 1 is the celebrity.

Example 2:

Input:
N = 2
M[][] = {{0 1},
{1 0}}
Output: -1
Explanation: The two people at the party both

know each other. None of them is a celebrity.

23. Given an array **arr[ ]** of size **N** having elements, the task is to find the next greater element for each element of the array in order of their appearance in the array.
Next greater element of an element in the array is the nearest element on the right which is greater than the current element.
If there does not exist next greater of current element, then next greater element for current element is -1. For example, next greater of the last element is always -1.

Example 1:

Input:
N = 4, arr[] = [1 3 2 4]
Output:
3 4 4 -1
Explanation:
In the array, the next larger element
to 1 is 3 , 3 is 4 , 2 is 4 and for 4 ?
since it doesn't exist, it is -1.

Example 2:

Input:
N = 5, arr[] [6 8 0 1 3]
Output:
8 -1 1 3 -1
Explanation:
In the array, the next larger element to
6 is 8, for 8 there is no larger elements
hence it is -1, for 0 it is 1 , for 1 it
is 3 and then for 3 there is no larger

element on right and hence -1.

Your Task:
This is a **function** problem. You only need to complete the function **nextLargerElement()** that takes list of integers **arr[ ]** and **N** as input parameters and returns list of integers of length N denoting the next greater elements for all the corresponding elements in the input array.

Expected Time Complexity : O(N)
Expected Auxiliary Space : O(N)

Constraints:
$1 \leq N \leq 10^6$
$0 \leq A_i \leq 10^{18}$

24. Given an integer array. The task is to find the maximum of the minimum of every window size in the array.
Note: Window size varies from 1 to the size of the Array.

Example 1:

Input:
N = 7
arr[] = {10,20,30,50,10,70,30}
Output: 70 30 20 10 10 10 10
Explanation:
1.First element in output indicates maximum of minimums of all windows of size 1.

2.Minimums of windows of size 1 are {10},
{20}, {30}, {50},{10}, {70} and {30}.
Maximum of these minimums is 70.
3. Second element in output indicates
maximum of minimums of all windows of
size 2.
4. Minimums of windows of size 2
are {10}, {20}, {30}, {10}, {10}, and
{30}.
5. Maximum of these minimums is 30
Third element in output indicates
maximum of minimums of all windows of
size 3.
6. Minimums of windows of size 3
are {10}, {20}, {10}, {10} and {10}.
7.Maximum of these minimums is 20.
Similarly other elements of output are
computed.

Example 2:

Input:
N = 3
arr[] = {10,20,30}
Output: 30 20 10
Explanation: First element in output
indicates maximum of minimums of all
windows of size 1.Minimums of windows
of size 1 are {10} , {20} , {30}.
Maximum of these minimums are 30 and
similarly other outputs can be computed

Your Task:

The task is to complete the function **maxOfMin**() which takes the array arr[] and its size N as inputs and finds the maximum of minimum of every window size and returns an array containing the result.

Expected Time Complxity : O(N)

Expected Auxilliary Space : O(N)

Constraints:

$1 <= N <= 10^5$

$1 <= arr[i] <= 10^6$

25. Given a string S with repeated characters. The task is to rearrange characters in a string such that no two adjacent characters are the same.

Note: The string has only lowercase English alphabets and it can have multiple solutions. Return any one of them.

Example 1:

Input : str = "geeksforgeeks"

Output: 1

Explanation: All the repeated characters of the given string can be rearranged so that no adjacent characters in the string is equal. Any correct rearrangement will show a output of 1.

Example 2:

Input : str = "bbbbb"

Output: 0

Explanation: Repeated characters in the string cannot be rearranged such that there should not be any adjacent repeated character.

Your task :
You don't have to read input or print anything. Your task is to complete the function**rearrangeString()** which takes the string as input and returns the modified string. If the string cannot be modified return "-1".
Note: The generatedoutput is 1 if the string is successfully rearranged and is 0 if rearranging is not possible.

Expected Time Complexity : O(NlogN), N = length of String
Expected Auxiliary Space : O(number of english alphabets)

Constraints :
1 <= length of string <= 10^4

26. Given a Binary Tree. Check whether it is Symmetric or not, i.e. whether the binary tree is a **Mirror image of itself** or not.

Example 1:

Input:
```
      5
    /   \
   1     1
  /       \
 2         2
```
Output: True
Explanation: Tree is mirror image of
itself i.e. tree is symmetric

Example 2:

Input:
```
      5
```

```
    /   \
   10     10
  / \     \
 20  20     30
```

Output: False

Your Task:

You don't need to read input or print anything. Your task is to complete the function **isSymmetric()** which takes the root of the Binary Tree as its input and returns True if the given Binary Tree is the same as the Mirror image of itself. Else, it returns False.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(Height of the Tree).

Constraints:
0<=Number of nodes<=100

27. Given a Binary Tree. Find the Zig-Zag Level Order Traversal of the Binary Tree.

Example 1:

Input:

```
      1
    /   \
   2     3
  / \   / \
 4   5 6   7
```

Output:
1 3 2 4 5 6 7

Example 2:

Input:

```
        7
      /   \
     9     7
    / \   /
   8   8 6
  / \
 10  9
```

Output:

7 7 9 8 8 6 9 10

Your Task:

You don't need to read input or print anything. Your task is to complete the function **zigZagTraversal()** which takes the root node of the Binary Tree as its input and returns a list containing the node values as they appear in the Zig-Zag Level-Order Traversal of the Tree.

Expected Time Complexity: O(N).

Expected Auxiliary Space: O(N).

Constraints:

$1 <= N <= 10^4$

28. Given a binary tree, find its height.

Example 1:

Input:

```
    1
   / \
  2   3
```

Output: 2

Example 2:

Input:

```
 2
  \
   1
  /
 3
```

Output: 3

Your Task:

You don't need to read input or print anything. Your task is to complete the function **height()** which takes root node of the tree as input parameter and returns an integer denoting the height of the tree. If the tree is empty, return 0.

Expected Time Complexity: O(N)
Expected Auxiliary Space: O(N)

Constraints:

1 <= Number of nodes <= 10^5
1 <= Data of a node <= 10^9

29. Given two binary trees, the task is to find if both of them are identical or not.

Example 1:

Input:

```
   1        1
  / \      / \
 2   3    2   3
```

Output: Yes

Explanation: There are two trees both having 3 nodes and 2 edges, both trees are identical having the root as 1, left child of 1 is 2 and right child of 1 is 3.

Example 2:

Input:

```
   1       1
  / \     / \
 2   3   3   2
```

Output: No

Explanation: There are two trees both having 3 nodes and 2 edges, but both trees are not identical.

Your task:

Since this is a functional problem you don't have to worry about input, you just

have to complete the function **isIdentical()** that takes two roots as parameters and returns true or false. The printing is done by the driver code.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(Height of the Tree).

Constraints:
1 <= Number of nodes <= 10^5
1 <=Data of a node <= 10^5

30. There are given **N** ropes of different lengths, we need to connect these ropes into one rope. The cost to connect two ropes is equal to sum of their lengths.
The task is to connect the ropes with minimum cost. Given **N** size array **arr[]** contains the lengths of the ropes.

Example 1:

Input:
n = 4
arr[] = {4, 3, 2, 6}
Output:
29
Explanation:
We can connect the ropes in following ways.
1) First connect ropes of lengths 2 and 3.
Which makes the array {4, 5, 6}. Cost of
this operation 2+3 = 5.
2) Now connect ropes of lengths 4 and 5.
Which makes the array {9, 6}. Cost of
this operation 4+5 = 9.
3) Finally connect the two ropes and all

ropes have connected. Cost of this
operation 9+6 =15
Total cost for connecting all ropes is 5
+ 9 + 15 = 29. This is the optimized cost
for connecting ropes.
Other ways of connecting ropes would always
have same or more cost. For example, if we
connect 4 and 6 first (we get three rope of 3,
2 and 10), then connect 10 and 3 (we get
two rope of 13 and 2). Finally we
connect 13 and 2. Total cost in this way
is 10 + 13 + 15 = 38.

Example 2:

Input:
n = 5
arr[] = {4, 2, 7, 6, 9}
Output:
62
Explanation:
First, connect ropes 4 and 2, which makes
the array {6,7,6,9}. Cost of
this operation 4+2 = 6. Next, add ropes
6 and 6, which results in {12,7,9}.
Cost of this operation 6+6 = 12.
Then, add 7 and 9, which makes the array {12,16}.
Cost of this operation 7+9 = 16. And
finally, add these two which gives {28}.
Hence, the total cost is 6 + 12 + 16 +
28 = 62.

Your Task:
You don't need to read input or print anything. Your task is to complete the function **minCost()** which takes an integer array arr[] and an integer n as arguments and returns the minimum cost.

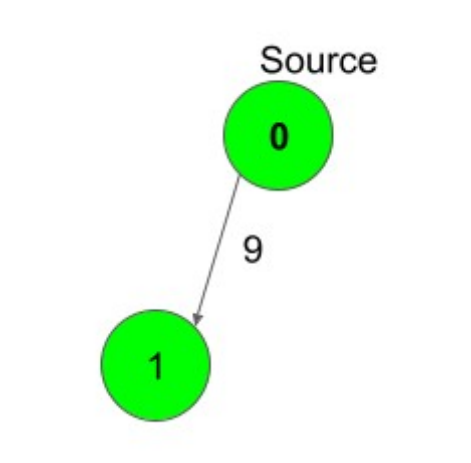
Expected Time Complexity : O(nlogn)
Expected Auxilliary Space : O(n)

Constraints:
$1 \leq N \leq 200000$
$1 \leq arr[i] \leq 10^6$

31. Given a weighted, directed and connected graph of V vertices and E edges, Find the shortest distance of all the vertex's from the source vertex S. If a vertices can't be reach from the S then mark the distance as 10^8. Note: If the Graph contains a negative cycle then return an array consisting of only -1.

Example 1:

Input:



E = [[0,1,9]]

S = 0

Output:

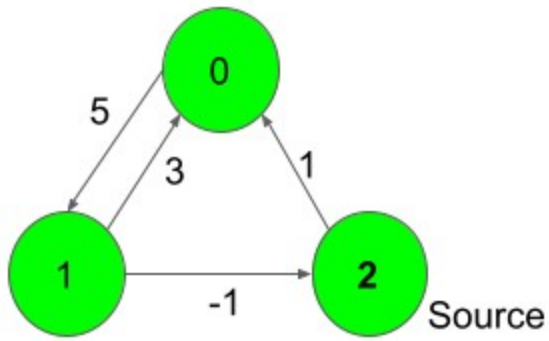
0 9

Explanation:

Shortest distance of all nodes from source is printed.

Example 2:

Input:



E = [[0,1,5],[1,0,3],[1,2,-1],[2,0,1]]

S = 2

Output:

1 6 0

Explanation:

For nodes 2 to 0, we can follow the path-2-0. This has a distance of 1.

For nodes 2 to 1, we cam follow the path-2-0-1, which has a distance of 1+5 = 6,

Your Task:

You don't need to read input or print anything. Your task is to complete the function **bellman_ford( )** which takes a number of vertices **V** and an **E**-sized list

of lists of three integers where the three integers are **u,v**, and **w**; denoting there's an edge from **u to v**, which has a weight of **w** and source node **S** as input parameters and returns a list of integers where the ith integer denotes the distance of an ith node from the source node.

If some node isn't possible to visit, then its distance should be 100000000(1e8). Also, If the Graph contains a negative cycle then return an array consisting of a single -1.

Expected Time Complexity: O(V*E).
Expected Auxiliary Space: O(V).

Constraints:
$1 \leq V \leq 500$
$1 \leq E \leq V*(V-1)$
$-1000 \leq adj[i][j] \leq 1000$
$0 \leq S < V$

32. Given two strings, one is a text string and the other is a pattern string. The task is to print the indexes of all the occurrences of the pattern string in the text string. For printing, Starting Index of a string should be taken as 1.

Example 1:

Input:
S = "batmanandrobinarebat", pat = "bat"
Output: 1 18
Explanation: The string "bat" occurs twice in S, one starts are index 1 and the other at index 18.

Example 2:

Input:
S = "abesdu", pat = "edu"
Output: -1
Explanation: There's not substring "edu"
present in S.

Your Task:
You don't need to read input or print anything. Your task is to complete the function **search()** which takes the string S and the string pat as inputs and returns an array denoting the start indices (**1-based**) of substring pat in the string S.

Note: You don't need to return -1 yourself when there are no possible answers, just return an empty list.

Expected Time Complexity: O(|S|).
Expected Auxiliary Space: O(|S|).

Constraints:
$1 \leq |S| \leq 10^5$
$1 \leq |pat| \leq |S|$

33. A **binary heap** is a Binary Tree with the following properties:
1) Its a *complete tree* (All levels are completely filled except possibly the last level and the last level has all keys as left as possible). This property of Binary Heap makes them suitable to be stored in an array.

2) A Binary Heap is either **Min Heap** or **Max Heap**. In a *Min Binary Heap*, the key at the *root* must be *minimum* among all keys present in Binary Heap. The same

property must be recursively true for all nodes in Binary Tree. Max Binary Heap is similar to MinHeap.

You are given an empty Binary Min Heap and some queries and your task is to implement the three methods **insertKey, deleteKey,** and **extractMin** on the Binary Min Heap and call them as per the query given below:

1) *1 x* (a query of this type means to insert an element in the min-heap with value x)

2) *2 x* (a query of this type means to remove an element at position x from the min-heap)

3) *3* (a query like this removes the min element from the min-heap and prints it).

Example 1:

Input:
Q = 7
Queries:
insertKey(4)
insertKey(2)
extractMin()
insertKey(6)
deleteKey(0)
extractMin()
extractMin()
Output: 2 6 - 1
Explanation: In the first test case for query
insertKey(4) the heap will have {4}
insertKey(2) the heap will be {2 4}
extractMin() removes min element from heap ie 2 and prints it

```
        now heap is {4}
insertKey(6) inserts 6 to heap now heap
        is {4 6}
deleteKey(0) delete element at position 0
        of the heap,now heap is {6}
extractMin() remove min element from heap
        ie 6 and prints it  now the
        heap is empty
extractMin() since the heap is empty thus
        no min element exist so -1
        is printed.
```

Example 2:

Input:
Q = 5
Queries:
insertKey(8)
insertKey(9)
deleteKey(1)
extractMin()
extractMin()
Output: 8 -1

Your Task:
You are required to complete the 3 methods **insertKey()** which take one argument the value to be inserted, **deleteKey()** which takes one argument the position from where the element is to be deleted and **extractMin()** which returns the minimum element in the heap(-1 if the heap is empty)

Expected Time Complexity: O(Q*Log(size of Heap)).
Expected Auxiliary Space: O(1).

Constraints:

$1 <= \mathbf{Q} <= 10^4$

$1 <= \mathbf{x} <= 10^4$

33. Implement a Queue using two stack **s1** and **s2**.

Example 1:

Input:
enqueue(2)
enqueue(3)
dequeue()
enqueue(4)
dequeue()
Output: 2 3
Explanation:
enqueue(2) the queue will be {2}
enqueue(3) the queue will be {2, 3}
dequeue() the poped element will be **2**
the queue will be {3}
enqueue(4) the queue will be {3, 4}
dequeue() the poped element will be **3**.

Example 2:

Input:
enqueue(2)
dequeue()
dequeue()
Output: 2 -1

Your Task:

Since this is a function problem, you don't need to take inputs. You are required to complete the two methods **enqueue()** which takes an integer **'x'** as input denoting the element to be pushed into the queue and **dequeue()** which returns the integer poped out from the queue.

Expected Time Complexity: O(1) for **enqueue()** and O(n) for **dequeue()**
Expected Auxiliary Space: O(1) for both **enqueue()** and **dequeue()**

Constraints:
1 <= Number of queries <= 100
1 <= values of the stack <= 100

34. Trie is an efficient information retrieval data structure. Use this data structure to store Strings and search strings. Your task is to use TRIE data structure and search the given string A. If found print 1 else 0.

Example 1:

Input:
N = 8
key[] = {the,a,there,answer,any,by,
bye,their}
search = the
Output: 1
Explanation: the is present in the given string "the a there answer any by bye their"

Example 2:

Input:
N = 8

key[] = {the,a,there,answer,any,by,
bye,their}
search = geeks
Output: 0
Explanation: geeks is not present in the given string "the a there answer any by bye their"

Your Task:
Complete **insert** and **search** function and return **true** if key is present in the formed trie else **false** in the search function. (In case of true, 1 is printed and false, 0 is printed by the driver's code.

Expected Time Complexity: O(M+|search|).
Expected Auxiliary Space: O(M).
M = sum of the length of all strings which is present in the key[]

|search| denotes the length of the string search.

Constraints:
$1 <= N <= 10^4$
1 <= |input strings|, |A| <= 100

35. There is **one** meeting room in a firm. There are **N** meetings in the form of **(start[i], end[i])** where **start[i]** is start time of meeting **i** and **end[i]** is finish time of meeting **i.**
What is the **maximum** number of meetings that can be accommodated in the meeting room when only one meeting can be held in the meeting room at a particular time?

Note: Start time of one chosen meeting can't be equal to the end time of the other chosen meeting.

Example 1:

Input:
N = 6
start[] = {1,3,0,5,8,5}
end[] = {2,4,6,7,9,9}
Output:
4
Explanation:
Maximum four meetings can be held with
given start and end timings.
The meetings are - (1, 2),(3, 4), (5,7) and (8,9)

Example 2:

Input:
N = 3
start[] = {10, 12, 20}
end[] = {20, 25, 30}
Output:
1
Explanation:
Only one meetings can be held
with given start and end timings.

Your Task :
You don't need to read inputs or print anything. Complete the function **maxMeetings()** that takes two arrays **start[]** and **end[]** along with their size **N** as input parameters and returns the **maximum** number of meetings that can be held in the meeting room.

Expected Time Complexity : O(N*LogN)
Expected Auxilliary Space : O(N)

Constraints:
$1 \leq N \leq 10^5$
$0 \leq$ **start[i]** $<$ **end[i]** $\leq 10^5$

36. There are **N** piles of coins each containing **Ai** (1<=i<=N) coins. Find the minimum number of coins to be removed such that the absolute difference of coins in any two piles is at most **K.**
Note: You can also remove a pile by removing all the coins of that pile.

Example 1:

Input:
N = 4, K = 0
arr[] = {2, 2, 2, 2}
Output:
0
Explanation:
For any two piles the difference in the number of coins is <=0. So no need to remove any coins.

Example 2:

Input:
N = 6, K = 3

arr[] = {1, 5, 1, 2, 5, 1}

Output :

2

Explanation:

If we remove one coin each from both
the piles containing 5 coins , then
for any two piles the absolute difference
in the number of coins is <=3.

Your Task:

You don't need to read input or print anything. Your task is to complete the function **minSteps()** which takes 2 integers N, and K and an array A of size N as input and returns the minimum number of coins that need to be removed.

Expected Time Complexity: O(N*logN)
Expected Auxiliary Space: O(N)

Constraints:

$1 \le N \le 10^5$
$0 \le K \le 10^4$
$1 \le A[i] \le 10^3$

37. Given an array **arr[ ]** of length **N** consisting cost of **N** toys and an integer **K** depicting the amount with you. Your task is to find maximum number of toys you can buy with **K** amount.

Example 1:

Input:

N = 7
K = 50
arr[] = {1, 12, 5, 111, 200, 1000, 10}

Output: 4
Explaination: The costs of the toys
you can buy are 1, 12, 5 and 10.

Example 2:

Input:
N = 3
K = 100
arr[] = {20, 30, 50}
Output: 3
Explaination: You can buy all toys.

Your Task:
You do not need to read input or print anything. Your task is to complete the function **toyCount()** which takes the integer **N**, integer **K** and the array **arr[ ]** as input parameters and returns the maximum count of toys you can have.

Expected Time Complexity: O(N * Log(N))
Expected Auxiliary Space: O(1)

Constraints:
$1 \leq N \leq 10^5$
$1 \leq K, arr[i] \leq 10^9$

38. Given an array **arr[]** denoting heights of **N** towers and a positive integer **K.**

For **each** tower, you must perform **exactly one** of the following operations **exactly once**.

- **Increase** the height of the tower by **K**
- **Decrease** the height of the tower by **K**

Find out the **minimum** possible difference between the height of the shortest and tallest towers after you have modified each tower.

You can find a slight modification of the problem here.
Note: It is **compulsory** to increase or decrease the height by K for each tower. **After** the operation, the resultant array should **not** contain any **negative integers**.

Example 1:

Input:
K = 2, N = 4
Arr[] = {1, 5, 8, 10}
Output:
5
Explanation:
The array can be modified as
{1+k, 5-k, 8-k, 10-k} = {3, 3, 6, 8}.
The difference between
the largest and the smallest is 8-3 = 5.

Example 2:

Input:
K = 3, N = 5
Arr[] = {3, 9, 12, 16, 20}
Output:
11
Explanation:
The array can be modified as
{3+k, 9+k, 12-k, 16-k, 20-k} -> {6, 12, 9, 13, 17}.
The difference between

the largest and the smallest is 17-6 = 11.

Your Task:
You don't need to read input or print anything. Your task is to complete the function **getMinDiff()** which takes the **arr[], n,** and **k** as input parameters and returns an integer denoting the minimum difference.

Expected Time Complexity: O(N*logN)
Expected Auxiliary Space: O(N)
Constraints
$1 \le K \le 10^4$
$1 \le N \le 10^5$
$1 \le Arr[i] \le 10^5$

39. Geek lost the password of his super locker. He remembers the number of digits **N** as well as the sum **S** of all the digits of his password. He know that his password is the largest number of **N** digits that can be made with given sum **S**. As he is busy doing his homework, help him retrieving his password.

Example 1:

Input:
N = 5, S = 12
Output:
93000
Explanation:
Sum of elements is 12. Largest possible
5 digit number is 93000 with sum 12.

Example 2:

Input:

N = 3, S = 29

Output:

-1

Explanation:

There is no such three digit number
whose sum is 29.

Your Task :

You don't need to read input or print anything. Your task is to complete the function l**argestNumber()** which takes 2 integers **N** and **S** as input parameters and returns the password in the form of string, else return "-1" in the form of string.

Constraints:

$1 \leq N \leq 10^4$

$0 \leq S \leq 9*10^4$

Expected Time Complexity : O(N)

Expected Auxilliary Space : O(1)

Given *weights* and *values* of **N** items, we need to put these items in a knapsack of capacity **W** to get the *maximum* total value in the knapsack.

Note: Unlike 0/1 knapsack, you are allowed to break the item.

Example 1:

Input:

N = 3, W = 50

values[] = {60,100,120}

weight[] = {10,20,30}

Output:

240.00
Explanation: Total maximum value of item
we can have is 240.00 from the given
capacity of sack.

Example 2:

Input:
N = 2, W = 50
values[] = {60,100}
weight[] = {10,20}
Output:
160.00
Explanation:
Total maximum value of item
we can have is 160.00 from the given
capacity of sack.

Your Task :
Complete the function ***fractionalKnapsack()*** that receives maximum capacity , array of structure/class and size n and returns a double value representing the maximum value in knapsack.
Note: The details of structure/class is defined in the comments above the given function.

Expected Time Complexity : O(NlogN)
Expected Auxilliary Space: O(1)

Constraints:
$1 <= N <= 10^5$
$1 <= W <= 10^5$

40. Given a set of **N** jobs where each $\mathbf{job_i}$ has a deadline and profit associated with it.

Each job takes ***1*** unit of time to complete and only one job can be scheduled at a time. We earn the profit associated with job if and only if the job is completed by its deadline.

Find the number of jobs done and the **maximum profit**.

Note: Jobs will be given in the form (Job_{id}, Deadline, Profit) associated with that Job.

Example 1:

Input:
N = 4
Jobs = {(1,4,20),(2,1,10),(3,1,40),(4,1,30)}
Output:
2 60
Explanation:
Job_1 and Job_3 can be done with
maximum profit of 60 (20+40).

Example 2:

Input:
N = 5
Jobs = {(1,2,100),(2,1,19),(3,2,27),
(4,1,25),(5,1,15)}
Output:
2 127
Explanation:

2 jobs can be done with
maximum profit of 127 (100+27).

Your Task :
You don't need to read input or print anything. Your task is to complete the function **JobScheduling()** which takes an integer **N** and an array of Jobs(Job id, Deadline, Profit) as input and returns the count of jobs and maximum profit as a list or vector of 2 elements.

Expected Time Complexity: O(NlogN)
Expected Auxilliary Space: O(N)

Constraints:
$1 <= N <= 10^5$
1 <= Deadline <= N
1 <= Profit <= 500

41. Given an array of size **n** such that each element contains either a 'P' for policeman or a 'T' for thief. Find the maximum number of thieves that can be caught by the police.
Keep in mind the following conditions :

1. Each policeman can catch only one thief.
2. A policeman cannot catch a thief who is more than **K** units away from him.

Example 1:

Input:
N = 5, K = 1
arr[] = {P, T, T, P, T}
Output: 2

Explanation: Maximum 2 thieves can be caught. First policeman catches first thief and second police man can catch either second or third thief.

Example 2:

Input:
N = 6, K = 2
arr[] = {T, T, P, P, T, P}
Output: 3
Explanation: Maximum 3 thieves can be caught.

Your Task:
You dont need to read input or print anything. Complete the function **catchThieves()** that takes **n, k** and **arr[ ]** as input parameters and returns the maximum number of thieves that can be caught by the police.

Expected Time Complexity: O(N)
Expected Auxiliary Space: O(N)

Constraints:
$1 \le n \le 10^5$
$1 \le k \le 100$
arr[i] = 'P' or 'T'

42. A gallery with plants is divided into **n** parts, numbered : 0,1,2,3...n-1. There are provisions for attaching water sprinklers at every partition. A sprinkler with range **x** at partition **i** can water all partitions from **i-x** to **i+x**.
Given an array **gallery[ ]** consisting of **n** integers, where **gallery[i]** is the range of sprinkler at partition **i** (power==-1 indicates no sprinkler attached), return the minimum number of sprinklers that need to be turned on to water the complete

gallery.
If there is no possible way to water the full length using the given sprinklers, print -1.

Example 1:

Input:
n = 6
gallery[ ] = {-1, 2, 2, -1, 0, 0}
Output:
2
Explanation: Sprinklers at index 2 and 5
can water thefull gallery, span of
sprinkler at index 2 = [0,4] and span
â€‹of sprinkler at index 5 = [5,5].

Example 2:

Input:
n = 9
gallery[ ] = {2, 3, 4, -1, 2, 0, 0, -1, 0}
Output:
-1
Explanation: No sprinkler can throw water
at index 7. Hence all plants cannot be
watered.

Your task:
Your task is to complete the function **min_sprinklers()** which takes the array **gallery[ ]** and the integer **n** as input parameters and returns the value to be printed.

Expected Time Complexity: O(NlogN)
Expected Auxiliary Space: O(N)

Constraints:
$1 \leq n \leq 10^5$
gallery[i] ≤ 50

43. There are **n** stairs, a person standing at the bottom wants to reach the top. The person can climb either **1 stair or 2 stairs at a time**. Count the number of ways, the person can reach the top (**order does matter**).

Example 1:

Input:
n = 4
Output: 5
Explanation:
You can reach 4th stair in 5 ways.
Way 1: Climb 2 stairs at a time.
Way 2: Climb 1 stair at a time.
Way 3: Climb 2 stairs, then 1 stair
and then 1 stair.
Way 4: Climb 1 stair, then 2 stairs
then 1 stair.
Way 5: Climb 1 stair, then 1 stair and
then 2 stairs.

Example 2:

Input:
n = 10
Output: 89

Explanation:
There are 89 ways to reach the 10th stair.

Your Task:
Complete the function **countWays()** which takes the top stair number m as input parameters and returns the answer **% 10^9+7**.

Expected Time Complexity : O(n)
Expected Auxiliary Space: O(1)

Constraints:
$1 \leq n \leq 10^4$

44. Given a number N. Find the minimum number of squares of any number that sums to N. For Example: If N = 100 , N can be expressed as (10*10) and also as (5*5 + 5*5 + 5*5 + 5*5) but the output will be 1 as minimum number of square is 1 , i.e (10*10).

Example 1:

Input: N = 100
Output: 1
Explanation: 10 * 10 = 100

Example 2:

Input: N = 6
Output: 3
Explanation = 1 * 1 + 1 * 1 + 2 * 2 = 6

Your Task:
You don't need to read or print anyhting. Your task is to complete the function **MinSquares()** which takes N as input parameter and returns minimum number of squares needed to make N.

Expcted Time Complexity: O(N * sqrt(N))
Expected Space Complexity: O(N)

Constraints:
1 <= N <= 10000

45. Given a positive integer **n**, find the nth fibonacci number. Since the answer can be very large, return the answer modulo **1000000007**.

Example 1:

Input:
n = 2
Output:
1
Explanation:
1 is the 2nd number of fibonacci series.

Example 2:

Input:
n = 5
Output:
5

Explanation:
5 is the 5th number of fibonacci series.

Your Task:
You dont need to read input or print anything. Complete the function **nthFibonacci()** which takes n as input parameter and returns nth fibonacci number.

Expected Time Complexity: O(n)
Expected Auxiliary Space: O(n)

Constraints:
$1 <= n <= 10^5$

46. You are given weights and values of **N** items, put these items in a knapsack of capacity **W** to get the maximum total value in the knapsack. Note that we have only **one quantity of each item**.
In other words, given two integer arrays **val[0..N-1]** and **wt[0..N-1]** which represent values and weights associated with **N** items respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of **val[]** such that sum of the weights of this subset is smaller than or equal to **W.** You cannot break an item, **either pick the complete item or dont pick it (0-1 property)**.

Example 1:

Input:
N = 3
W = 4
values[] = {1,2,3}

weight[] = {4,5,1}
Output: 3
Explanation: Choose the last item that weighs 1 unit and holds a value of 3.

Example 2:

Input:
N = 3
W = 3
values[] = {1,2,3}
weight[] = {4,5,6}
Output: 0
Explanation: Every item has a weight exceeding the knapsack's capacity (3).

Your Task:
Complete the function **knapSack()** which takes maximum capacity W, weight array wt[], value array val[], and the number of items n as a parameter and returns the **maximum possible** value you can get.

Expected Time Complexity: O(N*W).
Expected Auxiliary Space: O(N*W)

Constraints:
$1 \le N \le 1000$
$1 \le W \le 1000$
$1 \le wt[i] \le 1000$
$1 \le v[i] \le 1000$

47. Given a value **V** and array **coins[]** of size **M**, the task is to make the change for **V** cents, given that you have an infinite supply of each of coins{$coins_1$, $coins_2$, ..., $coins_m$} valued coins. Find the minimum number of coins to make the change. If not possible to make change then return -1.

Example 1:

Input: V = 30, M = 3, coins[] = {25, 10, 5}
Output: 2
Explanation: Use one 25 cent coin
and one 5 cent coin

Example 2:

Input: V = 11, M = 4,coins[] = {9, 6, 5, 1}
Output: 2
Explanation: Use one 6 cent coin
and one 5 cent coin

Your Task:
You don't need to read input or print anything. Complete the function **minCoins()** which takes **V, M** and array **coins** as input parameters and returns the answer.

Expected Time Complexity: O(V*M)
Expected Auxiliary Space: O(V)

Constraints:
$1 \leq \mathbf{V*M} \leq 10^6$

48. Given two strings **s** and **t.** Return the minimum number of operations required to convert **s** to **t**.
The possible operations are permitted:

1. Insert a character at any position of the string.
2. Remove any character from the string.
3. Replace any character from the string with any other character.

Example 1:

Input:
s = "geek", t = "gesek"
Output: 1
Explanation: One operation is required
inserting 's' between two 'e's of s.

Example 2:

Input :
s = "gfg", t = "gfg"
Output:
0
Explanation: Both strings are same.

Your Task:
You don't need to read or print anything. Your task is to complete the function **editDistance()** which takes strings s and t as input parameters and returns the minimum number of operation to convert the string **s** to string **t**.

Expected Time Complexity: O(|s|*|t|)
Expected Space Complexity: O(|s|*|t|)

Constraints:
1 ≤ Length of both strings ≤ 100
Both the strings are in lowercase.

49. Given an integer **N** denoting the Length of a line segment. You need to cut the line segment in such a way that the cut length of a line segment each time is either **x** , **y** or **z**. Here x, y, and z are integers.
After performing all the cut operations, your **total number of cut segments must be maximum**.

Note: if no segment can be cut then return 0.

Example 1:

Input:
N = 4
x = 2, y = 1, z = 1
Output: 4
Explanation: Total length is 4, and the cut lengths are 2, 1 and 1. We can make maximum 4 segments each of length 1.

Example 2:

Input:
N = 5
x = 5, y = 3, z = 2
Output: 2
Explanation: Here total length is 5, and the cut lengths are 5, 3 and 2. We can make two segments of lengths 3 and 2.

Your Task:
You only need to complete the function **maximizeTheCuts()** that takes n, x, y, z as parameters and returns **max number cuts**.

Expected Time Complexity : O(N)

Expected Auxiliary Space: O(N)

Constraints

1 <= N, x, y, z <= 10^4

50. You are given a set of **N** types of rectangular 3-D boxes, where the ith box has height **h**, width **w** and length **l**. Your task is to create a stack of boxes which is as tall as possible, but you can only stack a box on top of another box if the **dimensions** of the 2-D base of the lower box are each strictly larger than those of the 2-D base of the higher box. Of course, you can rotate a box so that any side functions as its base.It is also allowable to use multiple instances of the same type of box. Your task is to complete the function **maxHeight** which returns the height of the highest possible stack so formed.

Note:

Base of the lower box should be strictly larger than that of the new box we're going to place. This is in terms of both length and width, not just in terms of area. So, two boxes with same base cannot be placed one over the other.

Example 1:

Input:
n = 4
height[] = {4,1,4,10}
width[] = {6,2,5,12}
length[] = {7,3,6,32}
Output: 60

Explanation: One way of placing the boxes is as follows in the bottom to top manner:
(Denoting the boxes in (l, w, **h**) manner)
(12, 32, **10**) (10, 12, **32**) (6, 7, **4**)
(5, 6, **4**) (4, 5, **6**) (2, 3, **1**) (1, 2, **3**)
Hence, the total height of this stack is
10 + 32 + 4 + 4 + 6 + 1 + 3 = 60.
No other combination of boxes produces a height greater than this.

Example 2:

Input:
n = 3
height[] = {1,4,3}
width[] = {2,5,4}
length[] = {3,6,1}
Output: 15
Explanation: One way of placing the boxes is as follows in the bottom to top manner:
(Denoting the boxes in (l, w, **h**) manner)
(5, 6, **4**) (4, 5, **6**) (3, 4, **1**), (2, 3, **1**)
(1, 2, **3**).
Hence, the total height of this stack is
4 + 6 + 1 + 1 + 3 = 15
No other combination of boxes produces a height greater than this.

Your Task:
You don't need to read input or print anything. Your task is to complete the function **maxHeight()** which takes three arrays height[], width[], length[], and N

as a number of boxes and returns the **highest possible stack height** which could be formed.

Expected Time Complexity : O(N*N)
Expected Auxiliary Space: O(N)

Constraints:
1<= N <=100
1<= l,w,h <=100